RELATÓRIO DE EXERCÍCIO PRÁTICO

Projeto: Segurança em Banco de Dados

Implementação de restrições de validação e controle de acesso por Views

Aluno	Samuel Carlos Correia
Curso	Análise e Desenvolvimento de Sistemas — 5º semestre
Turma	ADS — Noite
Plataforma	Neon — PostgreSQL

Objetivo: demonstrar como o próprio banco de dados pode impedir informações inválidas e limitar o acesso a dados sensíveis.

1. Introdução

Este projeto apresenta uma aplicação prática de segurança em banco de dados PostgreSQL utilizando a plataforma Neon. Foram implementadas duas medidas principais: restrições CHECK, responsáveis por validar os dados no momento da gravação, e uma VIEW, responsável por apresentar somente as colunas necessárias a um determinado usuário.

A proposta segue dois princípios importantes: integridade dos dados e menor privilégio. A integridade garante que as informações gravadas respeitem regras lógicas. O menor privilégio garante que cada usuário tenha acesso apenas ao que precisa para realizar sua função.

2. Exercício 1 — Restrições de validação

2.1 Criação da tabela de transações

A tabela transacoes representa operações financeiras entre uma conta de origem e uma conta de destino.

```
CREATE TABLE transacoes (
    id INT PRIMARY KEY,
    conta_origem INT,
    conta_destino INT,
    valor DECIMAL(10,2)
);
```

2.2 Restrição para impedir valores menores ou iguais a zero

```
ALTER TABLE transacoes
ADD CONSTRAINT chk_transacoes_valor_positivo
CHECK (valor > 0);
```

A restrição acima permite somente valores maiores que zero. Assim, valores negativos e o valor zero são recusados pelo banco, mesmo que uma validação da aplicação seja ignorada ou contornada.

2.3 Restrição para impedir contas iguais

```
ALTER TABLE transacoes
ADD CONSTRAINT chk_contas_diferentes
CHECK (conta_origem <> conta_destino);
```

Essa regra impede que uma transação tenha a mesma conta como origem e destino.

2.4 Testes realizados

```
-- Inserção válida
INSERT INTO transacoes
    (id, conta_origem, conta_destino, valor)
VALUES
    (2, 300, 400, 75.00);
```

Resultado: declaração executada com sucesso.

```
-- Valor negativo
INSERT INTO transacoes
    (id, conta_origem, conta_destino, valor)
VALUES
    (3, 100, 200, -50.00);
```

Resultado: erro da restrição chk_transacoes_valor_positivo. O registro foi bloqueado corretamente.

```
-- Valor igual a zero
INSERT INTO transacoes
    (id, conta_origem, conta_destino, valor)
VALUES
    (3, 100, 200, 0.00);
```

Resultado: erro da restrição chk_transacoes_valor_positivo. O registro foi bloqueado corretamente.

```
-- Contas de origem e destino iguais
INSERT INTO transacoes
    (id, conta_origem, conta_destino, valor)
VALUES
    (3, 100, 100, 80.00);
```

Resultado: erro da restrição chk_contas_diferentes. O registro foi bloqueado corretamente.

```
SELECT *
FROM transacoes;
```

3. Exercício 2 — View para ocultação estrutural

3.1 Criação da tabela de funcionários

```
CREATE TABLE funcionarios (
    id INT PRIMARY KEY,
    nome VARCHAR(100),
    departamento VARCHAR(50),
    cpf VARCHAR(11),
    salario DECIMAL(10,2)
);
```

A tabela contém informações de identificação e departamento, além de dados sigilosos, como CPF e salário.

3.2 Criação da View pública

```
CREATE VIEW vw_funcionarios_publico AS
SELECT
    nome,
    departamento
FROM funcionarios;
```

A View funciona como uma tabela virtual. Ela permite consultar apenas nome e departamento, ocultando as colunas cpf e salario.

3.3 Configuração das permissões

A role atendimento já existia no banco Neon. Por isso, não foi necessário recriá-la. O acesso à tabela original foi removido e a leitura foi liberada somente na View.

```
REVOKE ALL PRIVILEGES
ON funcionarios
FROM atendimento;

GRANT SELECT
ON vw_funcionarios_publico
TO atendimento;
```

Com essa configuração, atendimento não deve acessar diretamente a tabela funcionarios, mas pode consultar vw_funcionarios_publico.

3.4 Dados de teste e consulta da View

```
INSERT INTO funcionarios
    (id, nome, departamento, cpf, salario)
VALUES
    (1, 'Ana Souza', 'Atendimento', '12345678901', 2500.00),
    (2, 'Carlos Lima', 'Tecnologia', '98765432100', 5000.00);

SELECT *
FROM vw_funcionarios_publico;
```

Resultado observado: foram exibidos somente nome e departamento. CPF e salário não apareceram, comprovando a ocultação das informações sensíveis.

4. Consultas de verificação

4.1 Verificar a existência da View

```
SELECT table_name
FROM information_schema.views
WHERE table_schema = 'public'
  AND table_name = 'vw_funcionarios_publico';
```

Resultado: vw_funcionarios_publico.

4.2 Verificar a permissão do usuário

```
SELECT
    grantee,
    table_name,
    privilege_type
FROM information_schema.role_table_grants
WHERE grantee = 'atendimento';
```

O resultado deve indicar a permissão SELECT para atendimento na View vw_funcionarios_publico.

5. Conclusão

O projeto demonstrou que a segurança não deve depender exclusivamente do Front-End ou do Back-End. As restrições CHECK criaram uma camada de proteção no próprio esquema do banco, bloqueando valores inválidos e impedindo transações entre contas iguais.

A criação da View aplicou o princípio do menor privilégio, pois o usuário atendimento recebeu acesso somente às informações necessárias para sua função. Dessa forma, dados como CPF e salário permaneceram protegidos na tabela original.

Os erros retornados pelo Neon durante os testes não representam falhas do projeto. Eles são evidências de que as regras de segurança foram acionadas corretamente. Portanto, a solução atende aos objetivos de validação, integridade e controle de acesso em banco de dados.

Resumo da solução: CHECK protege a validade dos dados; VIEW protege a visibilidade dos dados; GRANT e REVOKE controlam o acesso.